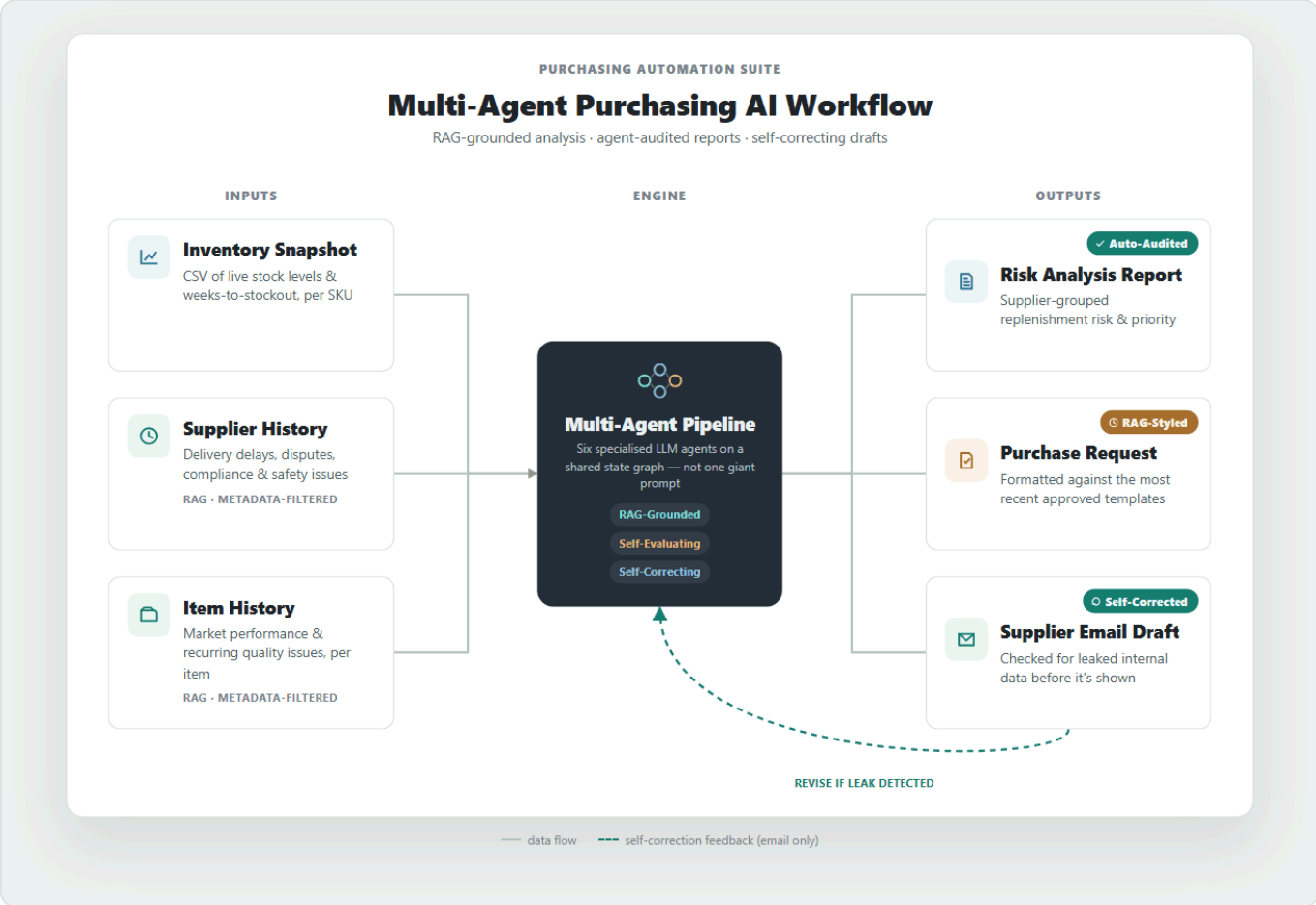
구매 의사결정 지원 멀티 에이전트 AI 시스템

(Multi-agent Purchasing AI Suite)

재고 스냅샷과 공급업체·아이템 히스토리 컨텍스트를 RAG로 함께 분석해, 구매 담당자들이 수작업으로 하던 **재고 리스크 분석·충당 계획 수립부터 분석 보고서·구매 요청서·공급업체 이메일 초안 작성까지**를 대신하는 멀티 에이전트 LLM 파이프라인.

특히 단순 재고 확인에 그치지 않고, 공급업체의 납기 준수·배송 지연 이력, 거래·협상 내역, 통관·규제·안전 이슈부터 해당 품목의 시장 퍼포먼스와 품목별 문제 이력까지 — **여러 문서에 흩어져 있던 관련 맥락을 RAG을 통해 자동으로 끌어와 분석**합니다. 생성된 결과물은 **별도 에이전트가 데이터·논리 일관성을 자체 검수해 품질 평가 보고서를 함께 생성**하며, 이메일 초안은 **내부 정보 유출 여부를 자동 점검해 문제가 있으면 스스로 재작성하는 자기 교정 루프**를 거칩니다.



데모: https://soominmyung.com/purchasing-automation

소스: https://github.com/soominmyung/purchasing-automation

프로젝트 개요

기간	2025년 10월 – 2025년 12월
역할	단독 프로젝트 — 아키텍처 설계, 개발, 배포
분야	생성형 AI 기반 재고 리스크 분석·문서 자동화
핵심 스택	Python, FastAPI, LangGraph, ChromaDB, GPT-4o, Docker, GCP Cloud Run
출발점	n8n 로우코드 프로토타입 → Python 서비스로 재설계

문제

구매 담당자는 수백 건의 SKU 단위 재고 데이터를 검토하고, 이를 토대로 분석 보고서·구매 요청서·공급업체 이메일 같은 문서를 작성하는 데 상당한 수작업을 들입니다.

문제는 이 과정이 **사람의 상황에 크게 좌우된다**는 점입니다. **바쁜 일정이나 담당자의 숙련도 등의 이슈로 인해** 공급업체의 납기·품질 이력, 과거 협상 조건, 통관·규제·안전 이슈, 품목의 시장 반응처럼 **여러 문서에 흩어진 근거 자료를 미처 다 확인하지 못하면** — 그 결과는 곧 **비효율적인 발주 및 잠재적 이윤 손실**로 이어집니다.

주요 기능

재고 스냅샷을 입력받으면 시스템이 네 가지 결과물을 자동으로 생성합니다.

1. **구매 분석 보고서** — 공급업체별로 그룹화된 재고 리스크 평가 및 충당 계획 수립
2. **품질 평가 보고서** — 별도 에이전트가 분석 결과의 데이터·논리 일관성을 감사
3. **구매 요청서(Purchasing Request, PR)** — 승인 절차용으로 공급업체별 서식화
4. **공급업체 이메일 초안** — 납기·재고 가용성 문의용 외부 커뮤니케이션

재고 스냅샷은 재고 부족이 우려되는 품목을 추린 CSV로 입력되고, 공급업체·아이템 이력 문서는 업로드해 벡터 DB에 임베딩한 뒤 분석 단계에서 자동으로 검색됩니다.

아키텍처

PoC 기반 전환(PoC-to-Production) — n8n 프로토타입에서 프로덕션 코드로

처음에는 **n8n** 로우코드 워크플로우로 개념을 빠르게 검증했습니다. 노드를 연결해 "CSV → 그룹화 → LLM 분석 → 문서 생성"의 흐름이 실제로 동작하는지 확인하는 단계였습니다.

검증이 끝난 뒤, 복잡한 분기 로직·상태 관리·컨테이너 배포를 구현하기 위해 전체를 **Python(FastAPI + LangGraph)**으로 재설계했습니다. 이 과정에서 노드 기반 흐름을 비동기 API와 상태 그래프로 옮기고, API 키 인증·스트리밍을 갖춘 배포 가능한 서비스로 다듬었으며, 공개 데모는 CI/CD(GitHub Actions → Cloud Run)까지 구성했습니다 (실제 운영은 회사 로컬 서버 구동).

멀티 에이전트 파이프라인 (LangGraph)

6개의 특화된 에이전트를 LangGraph 상태 그래프로 연결하고, 검증 노드에서 필요 시 이전 단계로 되돌아가 재작성합니다.

입력 **재고 스냅샷(CSV) + 공급업체·품목 컨텍스트**

01	**공급업체 그룹화 및 발주일·수량 계산** 리드 타임(18주) 역산 · 납기 후 26주 재고 커버리지 목표	결정론적 파이썬 산식, LLM 미사용
02	**분석 에이전트** 재고 리스크 분석	RAG: 메타데이터 필터링 후 최신 N개 이력 참고
03	**분석 평가 에이전트** 분석 결과 품질 감사 (논리·데이터 일관성)	정확성·충실도(할루시네이션)·추론·적합성 채점
04	**분석 보고서 작성 에이전트** 구매 분석 보고서 작성	RAG: 최신 업로드 순 상위 N개 모범 양식 참고
05	**PR 내용 생성 에이전트** 구매 요청서 내용 생성	RAG: 최신 업로드 순 상위 N개 모범 양식 참고
06	**PR 문서 작성 에이전트** 정식 구매 요청 문서 작성	RAG: 최신 업로드 순 상위 N개 모범 양식 참고
07	**이메일 에이전트** 공급업체용 이메일 초안	RAG: 최신 업로드 순 상위 N개 모범 양식 참고
08	**검증 노드** 민감정보(재고 현황, 분석 자료 등) 유출 검사 → 자동 재작성 루프	가드레일(민감정보 유출 방지), 실패 사유 피드백 기반 정밀 재작성 지시

왜 멀티 에이전트인가

하나의 거대한 에이전트 및 프롬프트로 전체를 처리하는 방식에 비해 다음과 같은 장점이 있습니다.

- **모듈화(Modularity)** — 각 단계를 서로 건드리지 않고 개별적으로 교체·개선할 수 있어, 실제로 분석 에이전트만 떼어 GPT-4o에서 자체 호스팅 Llama로 대체 가능성을 검증했습니다.
- **조기 오류 차단** — 분석 평가 에이전트가 분석 결과를 독립적으로 감사하기 때문에, 문제가 있는 분석이 뒤 단계(PR·이메일)로 그대로 전파되기 전에 걸러집니다.

- **정밀한 자기 교정 및 검증** — 검증 노드가 문제를 감지하면 전체 파이프라인을 처음부터 다시 돌릴 필요 없이, 문제가 생긴 단계만 정확히 지목해 재작성을 지시할 수 있습니다.

핵심 구성 요소

RAG 지식 베이스 (ChromaDB)

- **청킹·적재 구조** — 공급업체 기록·아이템 이력 PDF(개별 또는 ZIP 대량 업로드)를 청킹(1,000자, 200자 오버랩)해 OpenAI 임베딩으로 변환하고, 문서 유형별(공급업체 이력·품목 이력·보고서/PR/이메일 예시)로 분리된 5개의 독립 컬렉션에 저장합니다.
- **청크 사이즈·오버랩 튜닝** — 실제 공급업체·품목 이력 문서들이 A4 1~2페이지 분량으로 작성되는 것에 착안하여 청크 크기를 500자·1,000자·1,500자로 설정하여 저장·검색 효율을 비교 테스트했습니다. 너무 작은 청크 사이즈(500자)에 따른 문맥 소실, 너무 큰 청크 사이즈(1,500자)에 따른 문맥 뒤섞임 문제를 피해 최적의 청크 사이즈(1,000자)를 선택했고, 청크 경계에 걸친 문맥을 놓치지 않도록 20%(200자) 크기의 오버랩을 설정했습니다.
- **현업 워크플로우 맞춤형 설계** — 실제 공급업체·품목 이력 문서들이 서두에 공급업체명·아이템 코드를 명시하는 원칙을 따르는 것에 착안하여, 별도의 복잡한 온톨로지 구현 없이 **정규식만으로 신뢰할 수 있는 메타데이터 필터링**을 설계하였습니다.
  - **메타데이터 태깅** — 인제스트 시점에 정규식으로 공급업체명·아이템 코드를 추출해 문서에 메타데이터로 부여합니다. 청킹 이후에도 모든 하위 청크가 원본 문서의 메타데이터를 그대로 상속받으므로, 특정 청크의 텍스트에 그 이름이 등장하지 않아도 필터는 항상 정확하게 걸립니다.
  - **명시적 메타데이터 필터링 + 최신순 조회 (2단계 검색)** — 분석 에이전트가 히스토리를 조회할 때, 파이프라인이 이미 알고 있는 실제 공급업체명·아이템 코드를 메타데이터 필터로 전달해 그 공급업체·품목의 문서로만 후보군을 좁힙니다. 필터링만으로 이미 그 공급업체·품목에 정확히 해당하는 문서만 남기 때문에, 그 이후에는 유사도 계산 없이 **문서에 작성된 실제 기록 날짜(event_date) 기준 상위 N개**를 가져옵니다. 날짜 추출이 실패한 경우 인제스트 시점을 대신 사용합니다. 검색 목적에 따라 유연하게 다양한 필터링을 사용할 수 있습니다.
  - **강점** — 순수 시맨틱 유사도 검색만 쓰면 다른 공급업체의 유사한 서술(예: 비슷한 배송 지연 사례)이 섞여 들어올 위험이 있는데, 메타데이터 필터가 이런 교차 오염을 원천 차단합니다. 기존 업무 관행에 착안하여 온톨로지나 지식 그래프의 구현 없이도 이를 달성했다는 데 의의가 있습니다.
- **목적에 따른 차별화된 검색 전략 활용** — 공급업체·품목 이력 컬렉션과 예시(분석 보고서·PR·이메일 모범 양식) 컬렉션은 둘 다 문서 자체의 날짜(event_date) 기준 최신순으로 상위 N개를 가져옵니다. 차이는 **정렬 전에 필터링을 거치는지**이고, 이는 각 컬렉션을 무엇에 쓰는지에서 갈립니다.
  - **이력 — 필터링 후 정렬** — **특정 공급업체·품목에 대한 사실**을 찾는 검색이므로, 반드시 메타데이터로 후보를 좁힌 뒤 날짜순을 봐야 합니다.
  - **예시 — 컬렉션 전체에서 정렬** — 회사 서식이 최근 어떻게 바뀌었는지를 보는 **스타일 참고**라, 특정 대상으로 좁힐 필요가 없습니다.

  그래서 두 컬렉션 모두 인제스트 시점에 문서 내 날짜 표기를 파싱해 event_date로 남기고(파싱 실패 시 인제스트 시점으로 대체), **공급업체·품목 이력 쪽만 메타데이터 필터를 추가로 거치도록** 설계했습니다.
- **이력 조회 강제** — 조회 여부를 모델 판단에 맡기면, 입력에 이미 재고·품절 주수가 들어 있어 **이력이 있는데도 조회를 건너뛰는 누락**이 생기기 쉽습니다. 할루시네이션은 아니지만 근거를 빠뜨린 분석이라

오히려 발견하기 어렵습니다. 그래서 프롬프트에서 공급업체·품목 이력 조회를 각각 정확히 1회씩 **의무화**하고, 그 뒤로 조회 결과에 사건이 있으면 반드시 핵심 질문으로 남기고 결과가 없으면 "이력 없음"을 명시하도록 규칙을 이어 붙였습니다. 조회 자체를 건너뛰면 이후 장치들이 작동할 기회조차 사라지기 때문에, 이 강제가 근거 누락 방지 체계의 첫 고리에 해당합니다.

- **내용 구조화 에이전트와 문서화 에이전트의 역할 분리** — 분석→분석 보고서, PR 내용 생성→PR 문서 두 흐름 모두 **"내용을 구조화하는 에이전트"와 "그 결과를 정식 문서로 포맷팅하는 에이전트"를 분리하는 동일한 설계 원칙을 따릅니다. 하나의 프롬프트에 내용에 대한 추론(사고)과 문서 형식 준수(포맷팅)를 함께 맡기면 프롬프트가 무거워져 통제·유지보수가 어려워지고, 할루시네이션 관리도 힘들어지며, RAG 관점에서도 내용 판단용 참고와 형식 참고용이 한 프롬프트 안에 뒤섞여 복잡해집니다.**
  구체적으로 내용 구조화 에이전트(분석/PR 내용 생성)는 **"각 항목에 어떤 판단 기준·내용이 들어가야 하는지"**(정당화 사유, buyer 체크사항의 구체성)를 참고하고, 문서화 에이전트(분석 보고서/PR 문서)는 **"실제 문서가 어떻게 포맷되는지"**(헤더 구성, 문체)만 참고합니다. 내용 구조화 에이전트가 사례의 판단 기준이 아닌 내용 자체를 참조하지 않도록, 프롬프트에 "예시의 구조만 참고하고 실제 내용은 복사하지 말 것"을 명시했고, 배송 지연·가격 급등·품질 이슈 등 서로 다른 리스크 유형 케이스로 실제 테스트해 예시 내용이 섞이지 않았음을 확인했습니다.

AI 분석 품질 평가(Evaluator) 에이전트

별도 분석 평가 에이전트가 분석 에이전트의 출력(JSON)과 그 근거가 된 공급업체·품목 이력 원문을 대조해, **수치 정확성·근거자료 충실도(할루시네이션)·추론의 논리성·민감정보 유출 여부** 4개 기준으로 10점 만점 채점합니다. 재조회가 아니라 분석 에이전트가 실제로 참고했던 이력 원문을 그대로 전달받기 때문에, 두 에이전트가 동일한 근거를 놓고 판단합니다. 근거가 어긋나면 채점 기준 전반이 흔들리는데(예: 실제로는 근거가 있는 서술을 "출처 불명"으로 오판), 이를 원천적으로 방지합니다.

가드레일(Guardrail) — 자기 교정 루프

이메일 결과물은 검증 노드를 거치며 민감정보(재고 현황, 분석 자료 등) 유출 여부를 점검합니다. 유출이 감지되면 그대로 반환하지 않고 그래프가 초안 재생성으로 되돌아갑니다.

실시간 스트리밍 (SSE)

단일 요청-응답 대신, 파이프라인 각 단계(CSV 파싱 → 그룹화 → 분석 → 문서 생성)의 진행 상황을 Server-Sent Events로 클라이언트에 실시간 전송함으로써 사용자에게 "지금 어느 단계인지"를 실시간으로 보여주고, 오류 발생 시에도 즉시 알릴 수 있습니다.

하이브리드(Hybrid) 설계 — 결정론적 계산과 LLM 추론의 역할 분리

공급업체 그룹화 단계에서 `WksToOOS`(품절까지 남은 주 수)와 `CurrentStock`을 바탕으로 **권장 발주일과 수량을 파이썬 산식으로 계산**합니다. 명확한 산식으로 계산 가능한 값은 LLM 추론에 맡기지 않고 결정론적 코드로 처리해 할루시네이션 여지를 원천 제거했고, LLM은 그 결과를 받아 우선순위가 정리된 구조화 데이터를 대상으로 맥락 해석과 문서 작성에만 집중합니다. 계산을 LLM 호출에서 분리한 만큼 토큰 소모와 응답 지연도 줄어듭니다.

운영 비용 구조

공급업체 그룹 한 건을 처리할 때 7개 에이전트가 호출되며, 실제 시스템 프롬프트와 입출력을 OpenAI 토크나이저(`tiktoken`)로 계측한 결과는 다음과 같습니다(GPT-4o 기준 $2.50/$10.00 per 1M 토큰 가정).

에이전트	입력	출력	비용	비중
분석	4,606	611	$0.0176	29.1%
분석 평가	2,983	350	$0.0110	18.1%
분석 보고서	1,302	550	$0.0088	14.4%
PR 문서	1,222	550	$0.0086	14.1%
PR 내용 생성	1,242	450	$0.0076	12.5%
이메일 초안	1,406	350	$0.0070	11.6%
검증 (mini)	650	80	$0.0001	0.2%
합계	**13,411**	**2,941**	**$0.0607**	—

전체 입력 13,411토큰 중 **약 40%는 에이전트 간 재전달**(분석 결과를 후속 4개 에이전트에 넘기는 몫, 평가 에이전트가 분석과 동일한 이력 원문을 받는 몫)입니다. 이는 낭비가 아니라 **모듈성과 채점 정확성을 위해 지불하는 비용**으로, 제거하면 각각 단일 프롬프트 구조로의 회귀와 할루시네이션 오판을 초래합니다.

보안(Security) & 관측성(Observability)

보안: 공개 데모는 익명 사용자의 토큰 남용을 막기 위해 헤더 기반 API 키 인증(`X-API-Key`)과 IP 단위 요청 제한을 두었습니다. 사내 배포는 내부망 ID/PW 개인별 로그인을 구현했습니다. 관측성: LangSmith로 모든 노드의 토큰 사용량·지연·프롬프트 성능을 추적하고, LangChain 밖의 자체 로직도 `@traceable`로 계측합니다.

배포

공개 데모는 Docker 컨테이너로 GCP Cloud Run(서버리스, 스케일 투 제로)에 올려 누구나 접속할 수 있게 했고, CI/CD는 GitHub Actions로 구성해 `main` 브랜치에 푸시할 때마다 이미지를 빌드·자동 배포합니다. 실제 사내 운영에서는 SAP ERP가 상시 구동되는 24시간 로컬 서버에 앱을 상주시켜 내부망에서 팀이 사용하도록 구성했으며, 이 환경에서는 콜드 스타트나 벡터스토어 휘발성 문제가 없습니다.

파인튜닝 연구: Llama-3-8B에 SFT + DPO 적용

분석 에이전트는 매 요청마다 GPT-4o를 호출하기 때문에 **호출할 때마다 비용이 발생하고, 데이터도 외부로 전송됩니다**. 그래서 GPT-4o가 만든 분석 결과를 Llama-3-8B에 학습시키는 지식 증류(distillation)로, 자체 호스팅 모델이 GPT-4o를 대신할 수 있는지 검증했습니다.

학습 데이터

- **데이터 출처** — 실제 회사 데이터 대신 GPT-4o로 생성한 **합성 구매 시나리오**를 사용했습니다.
- **시나리오 구성** — 재고 행(품목코드·품목명·공급업체·리스크 등급·현재고·품절까지 주수)과 공급업체·품목 이력(자연어)으로 구성됩니다.
- **다양성** — 배송 지연·가격 급등·품질 이슈·수요 변동 같은 이력 패턴을 다양하게 담았습니다.
- **적대적(adversarial) 케이스 포함** — 깔끔한 데이터만 학습하면 정작 지저분한 실제 데이터 앞에서 무너지기 때문에, 판단이 까다로운 네 가지 유형을 의도적으로 섞었습니다. ① 현재고(`CurrentStock`)는 넉넉한데 품절까지 남은 주수(`WksToOOS`)는 짧아 두 수치가 서로 어긋나는 경우, ② 공급업체 이력에는 신뢰할 만한 업체로 적혀 있는데 품목 이력에는 그 업체의 납품 실패 기록이 남아 있어 두 이력이 상충하는 경우, ③ 이력 서술이 두루뭉술해 분석 근거로 삼기 어려운 경우, ④ 현재고나 리스크 등급 같은 값이 아예 비어 있는 경우입니다. 이런 데이터까지 학습한 모델은 억지로 답을 만들지 않고, **무엇이 모순되고 무엇이 빠졌는지 명시**합니다.
- **지식 증류(Knowledge Distillation)** — 이 시나리오에 대해 GPT-4o가 만든 분석 결과(JSON: 분석 보고서·핵심 질문·보충 타임라인)를 정답으로 삼아 Llama가 이를 재현하도록 학습했습니다.
- **파인튜닝 범위** — 파이프라인 전체가 아니라 대체하려는 **분석 에이전트의 출력**으로 한정했습니다.

1단계 — 지도 파인튜닝 (SFT)

- Llama-3-8B + QLoRA (4-bit, LoRA rank 16 — 학습 파라미터 약 41M / 8B, 0.51%)
- 교사 예제 30개 (GPT-4o가 시나리오별로 생성한 분석 JSON)
- **학습 데이터 형태**: JSONL, `{instruction, input: {inventory, supplier_history, item_history}, output: {analysis: {...}}}` — 정답 라벨은 `output.analysis`(분석 보고서·핵심 질문·보충 타임라인)만 사용
- Vertex AI, Tesla T4 1장, 5 epoch, 367초
- 학습 손실: 1.14 → 0.41

2단계 — 직접 선호 최적화 (DPO)

- 선호 쌍 25개 (홀드아웃 5개 제외)
- **선호 쌍 구성** — `chosen`은 GPT-4o 정답 analysis JSON으로 고정하고(지식 증류 목표상 모델 자신의 실력을 넘어서는 외부 기준점 필요), `rejected`는 SFT 모델이 동일 프롬프트에 대해 온도 0.8로 생성한 후보 4개를 GPT-4o judge로 채점해 그중 가장 낮은 점수를 받은 것만 선별했습니다. **judge에게는 문체·분량이 아닌 데이터 정확성·추론 품질만 평가하도록 명시해, 스타일 차이가 선호 신호에 섞이지 않도록** 했습니다.
- **학습 데이터(선호 쌍) 형태**: JSONL, `{prompt, chosen, rejected}` — `prompt`는 SFT와 동일한 템플릿(Response 제외), `rejected`**에는 판단 근거(채점 점수·코멘트)도 함께 기록해 사후 분석이 가능하도록 함**
- Vertex AI, Tesla T4, 3 epoch, 약 14분
- Unsloth `PatchDPOTrainer()` + TRL `DPOTrainer`

평가 (GPT-4o-as-judge, 홀드아웃 5개)

모델	평균 점수	JSON 유효율	채점 기준
Base Llama-3-8B	0.0 / 10	0%	GPT-4o가 다음 2개 기준을 각 10점 만점으로 채점한 뒤 평균: (1) **데이터 정확성** — 공급업체명·아이템 코드·재고·리스크 등급 등 입력값을 정확히 반영했는가, (2) **추론 품질** — 재고 충당 분석과 핵심 질문이 논리적으로 타당한가. (프로덕션 AI 분석 품질 평가 에이전트의 4개 기준과는 별개의, 이 벤치마크 전용 간이 채점 방식입니다.)
Llama-3-8B SFT	**9.5 / 10**	**100%**	
Llama-3-8B SFT + DPO	8.5 / 10	100%	
GPT-4o (기준)	10.0 / 10	100%	

결과 해석

- **SFT 성과** — GPT-4o 대비 약 95%의 평가 품질에 도달했고, 모든 예제에서 유효한 구조화 출력을 냈습니다. 자체 호스팅 모델로의 대체 가능성을 뒷받침하는 결과입니다.
- **DPO 성능 하락** — judge가 실제로 품질을 검증한 선호 쌍으로 학습했음에도 SFT 대비 오히려 하락(9.5 → 8.5)했습니다.
  - **원인 분석** — judge 코멘트를 보면 하락한 예제 대부분에서 "핵심 질문(critical questions) 항목이 빠졌다"는 지적이 반복됩니다. 학습 데이터를 뜯어보면 `chosen`(GPT-4o)은 25개 전부 이 필드를 포함하는 반면 `rejected`는 25개 중 7개(28%)만 포함하고 있어, 훈련 신호 대부분은 "이 필드를 포함하라"는 방향이어야 했습니다. 그런데도 결과가 반대로 나온 건, **DPO의 대조 학습이 시퀀스 전체의 상대적 확률만 조정할 뿐 "어느 부분이 원인인지"는 짚어주지 못하기 때문입니다(credit assignment 부재)**. 소수(7개)의 예외 사례가 노이즈로 섞여 들어가면서, 25개라는 적은 표본에서 의도와 다른 방향으로 일반화된 것으로 보입니다.
  - **결론** — 선호 쌍을 judge로 검증해도 이 규모(25쌍)에서는 DPO가 SFT 대비 안정적인 개선을 보장하지 못했습니다. **SFT가 이미 GPT-4o 대비 95% 품질에 도달한 상태를 감안하면, 남은 격차를 DPO로 메우려는 추가 투자보다 SFT 단독 채택이 이 프로젝트 규모에서는 더 합리적인 결론**입니다.

추론 서빙 및 양자화 (vLLM)

파인튜닝 연구로 자체 호스팅 가능한 분석 모델을 확보했다면, 이 단계에서는 **그 모델을 실제로 얼마나 효율적으로 서빙할 수 있는지**를 측정합니다 — GPT-4o로부터 자체 호스팅으로의 전환 여부를 좌우하는 핵심 요소입니다. 파인튜닝한 Llama-3-8B(base + LoRA)를 단일 **NVIDIA L4(24GB, GCP `g2-standard-8`)**에서 공식 **vLLM** OpenAI 호환 컨테이너로 서빙하고, 기존 평가에 쓰였던 순수 HuggingFace `generate()` 루프와 비교했습니다.

Continuous batching — 지연 증가 없이 처리량만 상승

동시 요청	처리량 (tok/s)	p50 지연 (s)
1	16.1	7.94
8	122.8	8.31
16	**242.2**	8.44
32	240.5	8.45

동시 요청을 1 → 16으로 올리면 **처리량이 약 15배(16 → 242 tok/s) 오르는데도 요청당 지연은 ~8초로 거의 그대로입니다** — 여러 요청을 GPU에서 병렬로 처리하는 continuous batching의 이점입니다. 약 16을 넘어서면 GPU 연산이 한계에 다다라, 동시 요청을 더 늘려도 처리량이 늘지 않습니다.

기존 추론 방식과의 비교, 그리고 FP8 양자화

- **기존 방식 대비 처리량** — 같은 GPU에서 기존 평가에 쓰던 HuggingFace `generate()` 순차 호출은 11 tok/s였지만, vLLM으로 서빙하자 처리량이 242 tok/s로 **약 22배** 늘었습니다.
- **FP8 양자화 적용** — 모델을 8비트로 로딩해(별도 변환 파일 없이 vLLM 옵션 하나로 적용, L4의 FP8 연산 유닛 활용) 처리량이 **393 tok/s**까지 올랐습니다. FP16 대비 **약 1.6배**이며, 가중치가 차지하던 메모리가 줄어든 만큼 KV 캐시 용량이 **21k → 81k 토큰(약 3.9배)**으로 늘어, 더 긴 문맥이나 더 많은 동시 요청을 감당할 여유가 생겼습니다.
- **양자화로 인한 품질 저하 확인** — 8비트 양자화 시 품질 저하 여부를 확인하기 위해, 시나리오 30개에 대한 FP16 출력과 FP8 출력을 이 프로젝트의 GPT-4o 평가 방식으로 채점하였습니다. 결과는 **7.2점 → 7.1점 (10점 만점, −0.1)**으로 차이가 없었습니다.

요점

GPU 한 장(L4)에서 처리량을 **11 → 242 → 393 tok/s, 총 약 36배**까지 끌어올렸습니다. FP8 양자화를 통해 품질 저하 없이 처리량과 메모리 여유를 함께 얻을 수 있었습니다. 따라서 로컬 모델을 통한 대체 가능성을 품질과 성능 측면에서 모두 확인했습니다.

전환 시 비용 구조 변화

파인튜닝 대상은 분석 에이전트 하나이므로, 로컬 전환의 효과도 그 항목에만 발생합니다.

항목	현재 (GPT-4o)	분석만 로컬 전환
분석 에이전트	$0.0176	**$0.00037** (약 1/47)
나머지 6개 에이전트	$0.0430	$0.0430 (변화 없음)
건당 합계	**$0.0607**	**$0.0434 (−28.5%)**

토큰 단가로는 약 47배 저렴하지만, **GPU는 사용량과 무관하게 상시 과금되는 고정비**입니다. 클라우드 L4 기준 월 약 $612이므로 **손익분기점은 월 35,000건(하루 약 1,200건)**이고, 이는 구매팀의 실제 사용량을 크게 웃돕니다.

비용 절감이 가능한 지점

- **프롬프트 캐싱** — 시스템 프롬프트가 매 호출 동일해 접두부 캐시 적중률이 높습니다. 구조 변경 없이 입력 비용의 일부를 줄일 수 있으나, 절감 폭은 전체의 10% 안팎입니다.
- **문서화 계열 에이전트의 모델 하향** — 보고서·PR·이메일은 형식 준수 비중이 커서 상위 모델이 과할 수 있습니다. 품질 검증을 전제로 `gpt-4o-mini` 등 하위 모델 적용을 검토할 수 있습니다.
- **복수 에이전트 동시 로컬화** — 이미 확인한 continuous batching으로 같은 GPU에 여러 에이전트 요청을 함께 태울 수 있습니다. 다른 에이전트도 함께 로컬화하면 고정비를 회수하고 경제성을 높일 수 있습니다. 파이프라인 전체를 전환하면 절감액이 커져 손익분기점이 약 3분의 1로 낮아집니다(하루 약 1,200건 → 340건).

자체 호스팅의 의의 — 데이터 보안

실제 사용 규모에서는 손익분기점에 도달하지 못하므로, 자체 호스팅을 정당화하는 것은 절감액이 아니라 **데이터가 외부로 전송되지 않는다는 점**입니다. 구매 이력·거래 조건·공급업체 평가처럼 민감한 정보를 다루는 워크플로우에서는 이쪽이 더 결정적인 요구사항이며, 비용은 부수적 이점에 가깝습니다. 다만 사내 상시 서버에 GPU를 증설하는 방식이라면 시간당 과금이 아니라 장비 상각으로 계산되어 손익분기점은 크게 낮아집니다.

※ 비용은 실제 프롬프트를 `tiktoken`으로 계측한 토큰 수에 공개 요율을 적용한 추정치이며, RAG 검색량과 출력 길이는 실측 평균을 사용했습니다. 요율은 변동될 수 있습니다.

성과

- **자동화 및 분석 품질 향상** — 반복적인 SKU 단위 재고 검토와 문서 작성을 자동화 — 담당자·상황에 따라 들쭉날쭉하던 판단을 일관된 품질로 끌어올림.
- **RAG 기반 컨텍스트 분석** — 사람이 하면 담당자의 분석 속도·숙련도에 따라 누락되기 쉬웠던 공급업체 이력·리드 타임 등 다중 소스 컨텍스트를, RAG로 몇 초 만에 모두 끌어와 결합함으로써 리스크 우선순위 기반 의사결정을 빠르고 일관되게 지원.
- **자체 호스팅 대체 가능성 검증** — 파인튜닝한 로컬 모델(SFT)이 GPT-4o 대비 약 95% 품질에 도달했고, vLLM 서빙으로 단일 GPU(L4)에서 순수 baseline 대비 약 36배 처리량까지 확보해 품질·성능 양쪽에서 대체 타당성 확인. 이번 검증은 분석 에이전트 한 곳에 한정되지만, 같은 방식을 나머지 에이전트로 확장하면 API 호출 비용 제거와 데이터 외부 미전송을 함께 달성 가능.
- **운영 지향 설계** — SSE 실시간 스트리밍(대기 중에도 진행 상황 실시간 확인, 타임아웃 리스크 회피), 자기 교정 에이전트 그래프(민감정보 유출 자동 검출·재작성), LangSmith 기반 관측성(노드별 토큰·지연·프롬프트 성능 및 비용 모니터링과 문제 원인 추적)을 갖춘 설계.
- **정확성과 신뢰성 확보** — 정확도가 중요한 계산(발주일·수량)은 결정론적 산식으로, 맥락 판단이 필요한 부분은 LLM 추론으로 역할을 분리.

- **현업 맞춤형 효율화 설계** — 실제 문서 작성 관행(공급업체명·아이템 코드를 서두에 명시)을 파악한 뒤, 온톨로지 같은 무거운 구조화 없이 정규식 메타데이터 추출 + 강제 필터링만으로 RAG 검색의 교차 오염(다른 공급업체·품목 문서가 섞여 들어오는 문제)을 해결. 일반적인 엔지니어링 패턴을 그대로 적용하기보다, 현업의 실제 작업 방식을 관찰해 거기 맞춘 가볍고 효율적인 해법을 설계.

향후 과제

GPT-4o → 자체 호스팅 모델 전환

현재 분석 에이전트는 GPT-4o 호출에 의존해 데이터가 외부로 전송되고 호출 비용도 발생합니다. 이를 줄이기 위해 자체 호스팅 모델을 검증했고, SFT만으로 GPT-4o 대비 약 95% 품질에 도달한 데 이어 vLLM 서빙으로 단일 GPU에서 충분한 처리량까지 확인해 **품질·성능 양쪽에서 대체 타당성을 확인**했습니다(Llama-3-8B 기준). DPO는 judge 검증까지 거친 선호 쌍으로 추가 실험했지만 이 규모에서는 SFT 대비 개선을 얻지 못해, 현재는 **SFT 단독**으로 전환을 진행합니다. 다만 추가로 개선할 여지는 다음과 같습니다.

- **DPO 재도전 조건** — 현재 규모에서는 SFT 단독이 합리적이지만, 학습 데이터가 늘어 남은 5% 격차를 좁힐 유인이 생기면 **DPO 적용을 다시 검토**할 수 있습니다. 이때는 ① **선호 쌍 규모를 늘리거나**, ② 기존 4기준 평가 에이전트를 재사용해 항목별로 선호 신호를 나누는 **다축 보상(multi-attribute reward)**을 적용하는 방법이 있습니다. 대조 학습 자체를 피하려면 좋은 후보만 골라 다시 학습하는 ③ **Best-of-N 필터링 + SFT**로 우회할 수도 있습니다.
- **최신 모델 활용 검토** — GPT-4o는 이 프로젝트에서 프로덕션 분석 에이전트(반복 호출, 비용 민감)이자 학습 데이터를 만드는 교사(일회성 호출, 비용 무관)라는 두 가지 역할을 겸합니다. 교사 쪽은 상위 모델로 바꿔도 일회성 비용이라 부담이 적어 학습 데이터 품질을 높이는 데 활용할 수 있고, 학생(대체) 쪽은 Llama-3-8B 외에 이후 공개된 Qwen, Llama 최신 버전 등 오픈소스 소형 모델을 비교 검증해 프로덕션 비용·성능을 함께 개선할 여지가 있습니다.
- **평가 방법의 한계와 보완 방향** — 이번 평가는 GPT-4o가 **교사(학습 데이터 생성)·정답(비교 기준)·심판(채점)**을 겸했으므로, 9.5/10은 "얼마나 정확한가"보다 **"GPT-4o를 얼마나 잘 재현하는가"**에 가깝습니다. 학습·평가 시나리오도 모두 합성 데이터 기반인데, 운영하면서 실제 사내 문서를 축적해 학습·평가에 활용하면 **합성 데이터로 근사했던 실제 문서의 표현과 현업의 판단 기준을 직접 반영**할 수 있습니다. 나아가 **교사와 다른 계열 모델로 채점**해 자기 선호 편향을 걷어내거나, 출력에 등장한 품목코드·수치가 입력에 실제로 존재하는지 **코드로 검증**해 심판이 개입하지 않는 지표를 함께 두는 방식이 가능합니다.

RAG 검색 확장 아이디어 — 온톨로지/지식 그래프 적용 검토

현재 방식(메타데이터 필터 + 최신순 조회)은 공급업체·품목 단위로 관련 문서를 빠르고 정확하게 찾아온다는 본래 목적에는 충분합니다. 여기서 한 걸음 더 나아가, "이 공급업체가 납품하는 다른 품목 중 과거 유사 품질 이슈가 있었던 것은?"처럼 **여러 문서를 넘나드는 관계·집계 질의까지 지원하고 싶다면**, 온톨로지/지식 그래프를 추가로 검토해볼 수 있습니다.

- **기술 접근 — 온톨로지 설계 → 지식 그래프(Knowledge Graph) 구현** — 먼저 "공급업체", "품목" 같은 엔티티 타입과 "납품한다", "품질 이슈가 있었다" 같은 관계 타입을 **온톨로지로 정의**합니다. 그다음 이 스키마를 따라 실제 데이터를 채운 **경량 지식 그래프(예: Neo4j)를 구축**해 기존 검색과 함께 씁니다. **지금 구현된 메타데이터 필터 + 최신순 조회가 "이 공급업체·품목에 대한 문서 찾기"를 그대로 맡고, "이**

공급업체와 연결된 다른 품목·이슈까지 온톨로지가 정의한 관계를 따라 찾아가기"는 새로 추가되는 지식 그래프가 맡는 식으로 역할을 나눕니다.

- **기대 효과 — 다중 홉 추론 및 집계 질의(Multi-hop Reasoning & Aggregation)** — 지금은 한 문서 안에서 답을 찾는 질문(이 공급업체 이력이 어떻다)에 강한데, 여러 문서·엔티티를 이어가야 답이 나오는 질문("이 공급업체가 납품하는 다른 품목 중에도 문제 있었던 게 있나?")이나, 셀 수 있는 질문("이번 분기 지연이 2번 이상인 공급업체는 몇 곳?")에도 정확하게 답할 수 있게 됩니다.
- **도입 조건 — 단계적 적용** — 온톨로지 설계(엔티티·관계 타입 정의)와 그 스키마에 맞춰 데이터에서 관계를 추출하는 작업이 추가로 필요합니다. 그러니 실제 사용 중에 이런 "여러 문서를 넘나드는 질문"이 자주 나온다는 게 확인된 다음에, 필요한 만큼만 점진적으로 붙이는 게 현실적입니다.

기술 스택

영역	기술
백엔드	Python, FastAPI (async, SSE)
오케스트레이션	LangGraph (상태 기반 멀티 에이전트), LangChain
LLM	GPT-4o / GPT-4o-mini; Llama-3-8B (QLoRA)
파인튜닝	Unsloth, TRL (SFTTrainer / DPOTrainer), Vertex AI, W&B
추론 서빙	vLLM (컨테이너, continuous batching), FP8 양자화, NVIDIA L4
벡터 DB	ChromaDB (RAG)
관측성	LangSmith
프런트엔드	React, TypeScript, Framer 커스텀 코드 컴포넌트
데이터 처리	Pandas, PyPDF, python-docx
인프라 / CI-CD	Docker, GCP Cloud Run, Vertex AI, Artifact Registry, GitHub Actions

---

명수민 · msm1640@gmail.com · 010-5600-4620